

Phase 3: Gmail Scanner and LLM Analysis

Technical documentation: CV storage, Gmail access, one merged Groq call per email, and the logic behind each module

Project	Job Agent (working name), AI job-search automation
Phase	3 of 9: Core scan logic
Prepared for	Naftali Murimi Njeri
Date	October 11, 2026
Builds on	Phase 1 (JWT authentication) and Phase 2 (Supabase, full schema)
Leads to	Phase 4: Telegram notifications for matches above 7/10

1. Purpose and Scope

Phases 1 and 2 gave the app accounts and a database. Phase 3 builds the first piece that does the actual job: it reads job-alert emails from Gmail, asks an AI model whether each one is a job advert and how well it matches the user's CVs, and saves the verdict. A scan is started by hand (a protected API call), so the logic can be tested against a real inbox before any automation exists.

Deliverables of this phase:

- **CV management.** Upload one or more CVs (PDF or text), list them, delete them. The text is extracted and stored.
- **Gmail access.** A one-time local login that produces a token, and a client that lists, fetches and parses emails.
- **One merged AI call per email.** Classification (job advert or not) and CV scoring happen in a single request to Groq, which returns structured JSON: category, job title, company, match score 1 to 10, the best-matching CV, and a short reason.
- **Scan orchestration.** Fetch, skip already-scanned emails, analyze the rest, save results, return a summary.
- **Results endpoint.** List scanned emails ranked by score.

Out of scope: Telegram messages (Phase 4), the React UI (Phase 5), per-user Gmail connections (Phase 6), CV refinement chat (Phase 7), and scheduled scans (Phase 8). Gmail access in this phase is a single local token for the developer's own mailbox.

2. What Was Built

File	Status	Responsibility
scripts/gmail_auth.py	new	One-time browser login to Google; writes token.json
app/agent/gmail_client.py	new	Loads and refreshes the Gmail token; lists message ids; fetches and parses one message
app/agent/analyzer.py	new	Builds the prompt, calls Groq, validates the reply (the merged classify + score call)
app/agent/scanner.py	new	Orchestrates a scan from start to summary
app/cv_store.py	new	Database queries for CVs, always scoped to a user
app/email_store.py	new	Database queries for scanned emails: duplicate check, save, ranked list
app/api/cvs.py	new	POST/GET/DELETE /cvs: upload with text extraction, list, delete
app/api/scan.py	new	POST /scan to run a scan; GET /scan/results for ranked results

File	Status	Responsibility
app/schemas/cv.py	new	CVResponse model (never exposes the CV text)
app/config.py	modified	LLM and Gmail settings added
app/main.py	modified	Registers the two new routers
.env / .env.example	modified	LLM_API_KEY, LLM_BASE_URL, LLM_MODEL, GMAIL_QUERY

No database changes were needed: the cvs and scanned_emails tables already existed from Phase 2.

3. Technologies and Packages

Item	Role in Phase 3
Groq (LLaMA 3.3 70B)	The AI model. Hosted, fast, free tier for development. Called through an OpenAI-compatible API
openai (Python package)	Only the client library. Pointing its base URL at Groq makes it talk to Groq; pointing it back at OpenAI needs no code change
Gmail API (read-only scope)	Source of the emails. The scope gmail.readonly can read but never send, change or delete mail
google-api-python-client, google-auth-oauthlib, google-auth-httplib2	Google's official libraries for the OAuth login flow and API calls
pypdf	Extracts text from uploaded PDF CVs
python-multipart	Lets FastAPI accept file uploads and form fields
beautifulsoup4	Strips HTML tags from emails that have no plain-text version

Installed in one command: `pip install openai pypdf python-multipart beautifulsoup4 google-api-python-client google-auth-oauthlib google-auth-httplib2`, followed by `pip freeze > requirements.txt`.

4. Architecture

The structure follows the same layering as Phase 2. Routes only translate HTTP into function calls. The scanner decides the order of work. Small single-purpose modules talk to the outside world: Gmail, Groq and Supabase.

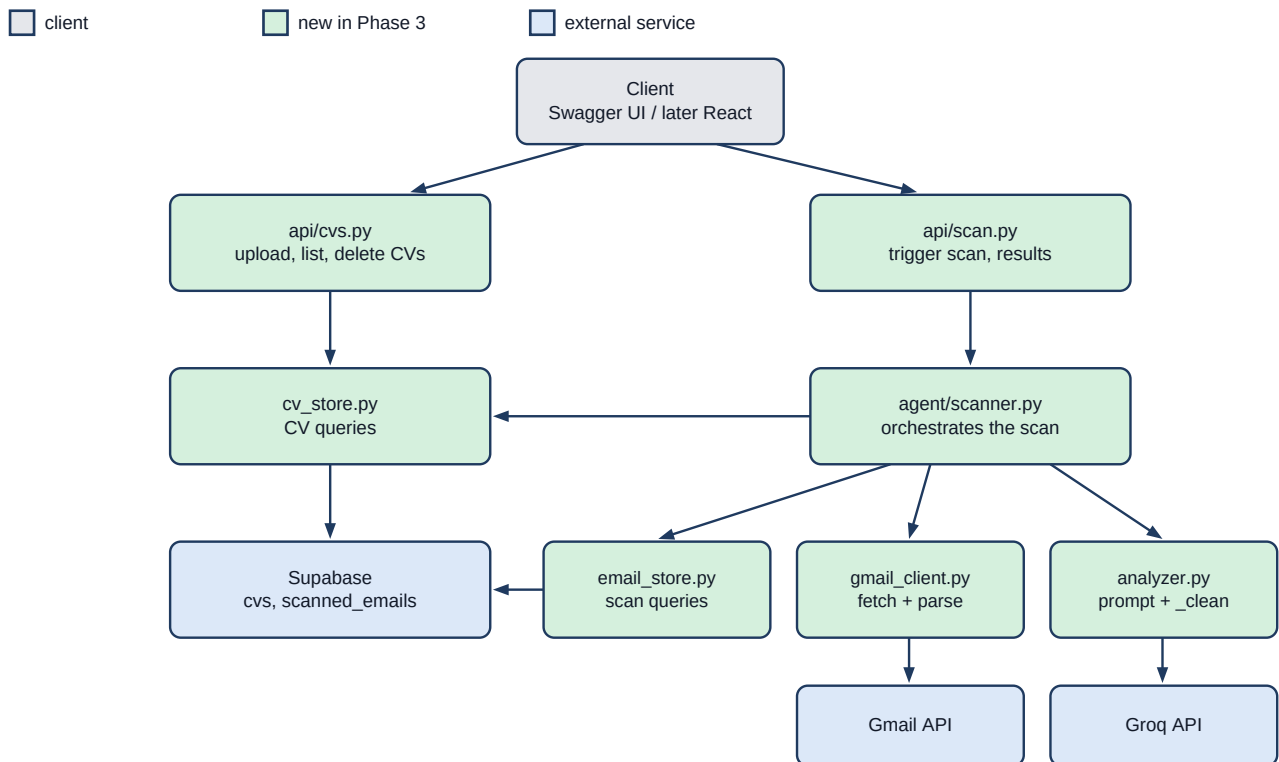


Figure 1: Components of Phase 3. Each arrow is a function call; the bottom row crosses the network.

Authentication is not redrawn: both routers depend on `get_current_user` from Phase 1, so every call here is already tied to a logged-in user whose id is passed down to the stores.

Design decisions

Decision	Reason
One merged AI call per email	The old build made two calls (classify, then score). One call halves the cost and latency and keeps the verdict consistent, because the model sees the email and all CVs together
Compare against ALL CVs, no manual tagging	The user tailors a CV per role (for example AI versus data analysis). The model picks the best fit and reports which CV won
CVs labelled "CV 1, CV 2..." in the prompt	The model answers with a small number, and code maps it to the real database id. Models can garble long UUIDs; numbers are safe
JSON mode plus a validator (<code>_clean</code>)	LLaMA 3.3 on Groq does not support strict JSON schemas. The format is described in the prompt and every field is re-checked in code before it reaches the database
OpenAI-compatible client with settings in <code>.env</code>	Provider, URL and model are configuration. Moving from Groq to OpenAI later is an <code>.env</code> edit only
Manual scan trigger	Fast feedback loop and no scheduler yet. Phase 8 wraps the same function in an Azure Functions timer
Local <code>token.json</code> for Gmail	Simplest way to test against a real inbox. Phase 6 replaces the token source with per-user connections; the rest stays
Save non-job emails too	A row with <code>is_job_advert = false</code> marks the email as seen, so it is never paid for twice
Failed emails are not saved	An email that errors out is simply retried on the next scan
Pause between emails	A short delay (<code>LLM_DELAY_SECONDS</code> , default 2) keeps requests under free-tier rate limits
Truncate inputs	Each CV is cut to 6,000 characters and each email body to 4,000, which bounds tokens per call
Plain def endpoints	The Supabase and Gmail libraries are synchronous; FastAPI runs sync endpoints in a thread pool so the server stays responsive

5. Use Case Diagram

The **User / API Client** (Swagger UI today, the React app in Phase 5) starts five use cases. Two of them involve other use cases that run inside the system. Three **external systems** take part: Gmail provides the mail, Groq provides the AI verdict, and Supabase stores everything.

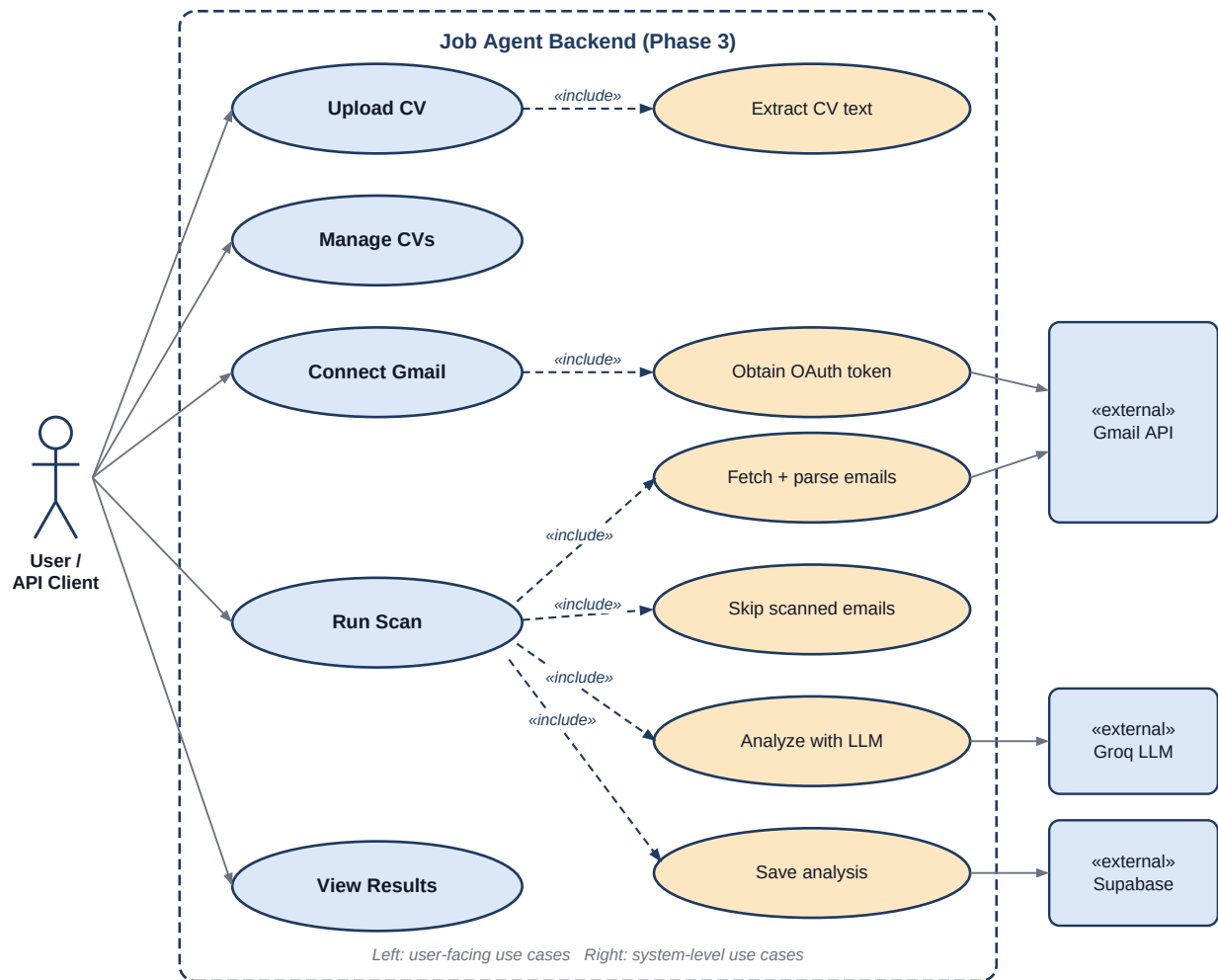


Figure 2: Use case diagram for Phase 3.

ID	Use case	Actors	Description and outcome
UC1	Upload CV	User, Supabase	Sends a name and a PDF or TXT. System checks size and type, extracts text (UC6), stores the CV. Result: 201 with CV details. The text itself is never returned
UC2	Manage CVs	User, Supabase	Lists the user's CVs or deletes one. Every query is filtered by user id, so users cannot see each other's CVs
UC3	Connect Gmail	User, Gmail	Run once from a terminal. Opens a browser consent screen and saves token.json (UC7). Must be repeated when the Google test-mode token expires (about 7 days)
UC4	Run Scan	User, Gmail, Groq, Supabase	POST /scan. Fetches recent emails (UC8), skips known ones (UC9), analyzes each new one (UC10), saves results (UC11). Result: a summary of counts
UC5	View Results	User, Supabase	GET /scan/results. Returns saved analyses, highest score first, optionally above a minimum score
UC6	Extract CV text	system	Included by UC1. pypdf for PDFs, UTF-8 decoding for text files. Rejects files with under 100 characters of text
UC7	Obtain OAuth token	Gmail	Included by UC3. Google returns an access token and a long-lived refresh token

ID	Use case	Actors	Description and outcome
UC8	Fetch + parse emails	Gmail	Included by UC4. Lists message ids for a search query, then reads each message and extracts headers and readable body text
UC9	Skip scanned emails	Supabase	Included by UC4. Compares fetched ids against scanned_emails so nothing is processed twice
UC10	Analyze with LLM	Groq	Included by UC4. One call returns classification, score and best CV for one email
UC11	Save analysis	Supabase	Included by UC4. Inserts one scanned_emails row per analyzed email

6. Data Flow Diagrams

6.1 Level 0: Context diagram

The backend is a single process. It exchanges data with the user, with Gmail, with Groq and with the database. Nothing goes to Groq except the text of one email and the CVs, which is worth knowing for privacy.

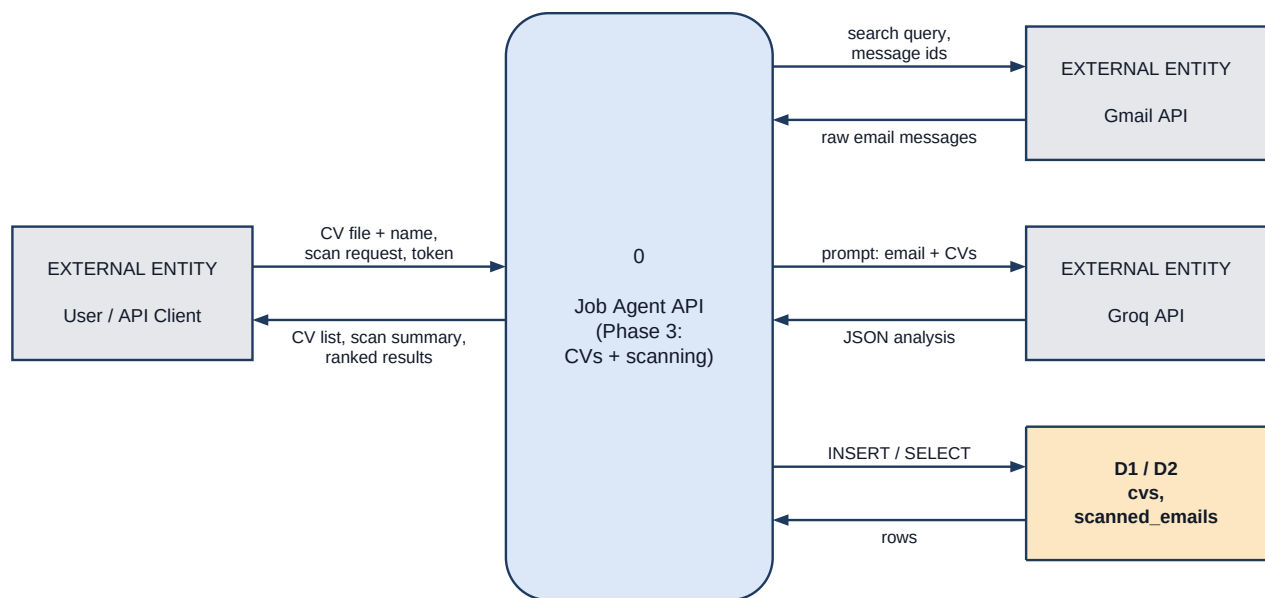


Figure 3: DFD level 0 (context). Rectangle = external entity, rounded box = process, open box = data store.

6.2 Level 1: Processes and data stores

Level 1 separates three concerns. **1.0 Manage CVs** only touches D1. **2.0 Run Scan** is the busy one: it reads CVs from D1, talks to Gmail and Groq, and reads and writes D2. **3.0 Query Results** is a pure read of D2.

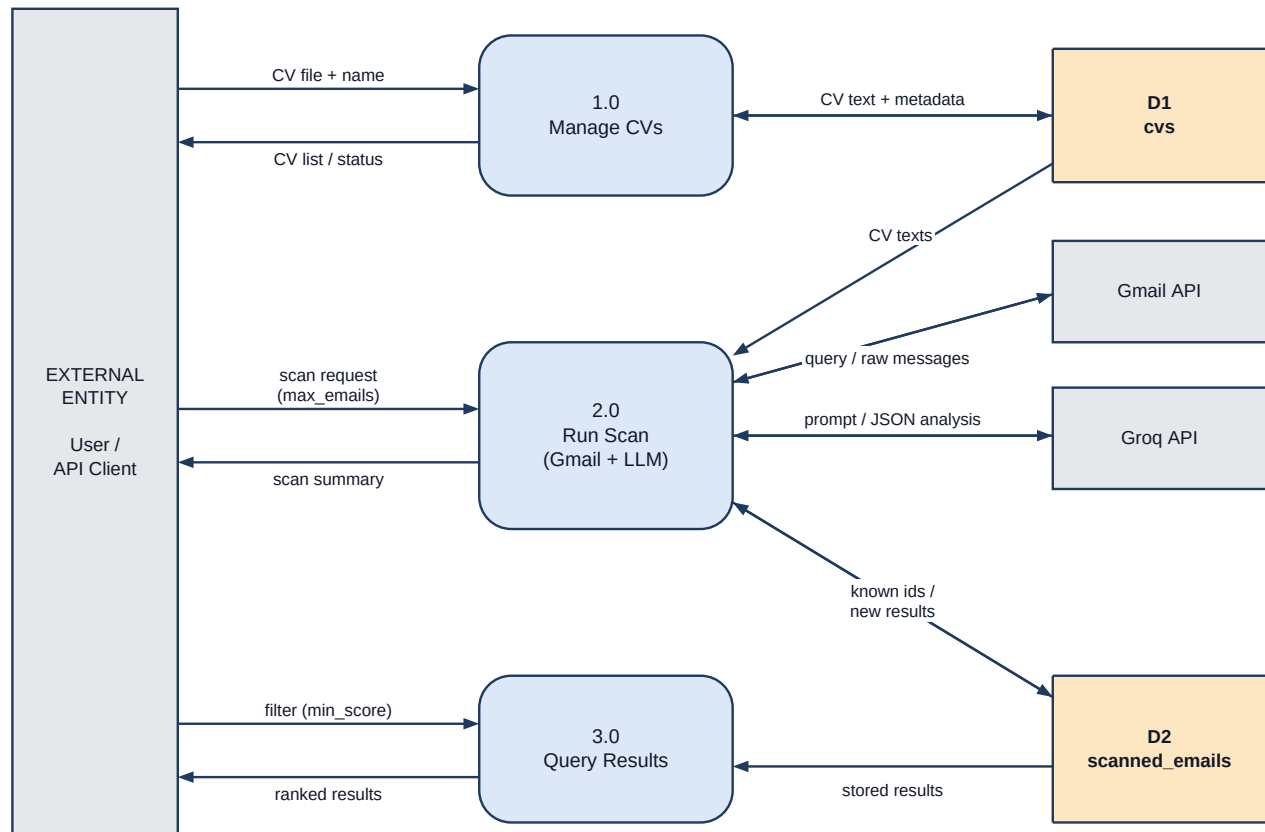


Figure 4: DFD level 1. Double-headed arrows are request and response pairs.

6.3 Data dictionary

Data flow	Contents	Notes
CV file + name	Multipart form: name, file (.pdf or .txt, up to 5 MB)	Sent with a Bearer token
CV text + metadata	user_id, name, file_name, content_text	content_text stays inside the backend; API responses omit it
Scan request	max_emails (1 to 50, default 10)	Query parameter on POST /scan
Search query	GMAIL_QUERY, default newer_than:3d	Setting in .env; returns up to max_emails ids
Raw email message	Headers (Subject, From), internalDate, MIME parts	Body may be plain text, HTML or both
Prompt: email + CVs	System prompt, numbered CVs (6,000 chars each), email (4,000 chars)	About 6,000 tokens for three CVs
JSON analysis	is_job_advert, category, job_title, company, match_score, best_cv_number, reasoning	Validated by _clean before use
Known ids / new results	gmail_message_ids already stored; one new row per analyzed email	Unique on (user_id, gmail_message_id)
Scan summary	fetched, already_scanned, analyzed, job_adverts, above_threshold, errors	Returned to the caller of POST /scan
Ranked results	Saved analyses ordered by match_score descending, nulls last	Optional min_score and limit filters

7. Activity Diagrams

Activity diagrams show the order of steps and every decision. Rounded boxes are actions, diamonds are decisions, orange boxes are early exits, and the filled circle and ringed circle mark start and end.

7.1 Upload a CV

Validation happens in cheapest-first order: authenticate, check size, check type, then do the work of extracting text. A PDF that cannot be parsed at all is also answered with 400 inside the extraction step. The 100-character floor catches scanned (image-only) PDFs, which contain no extractable text and would produce an empty CV for the AI.

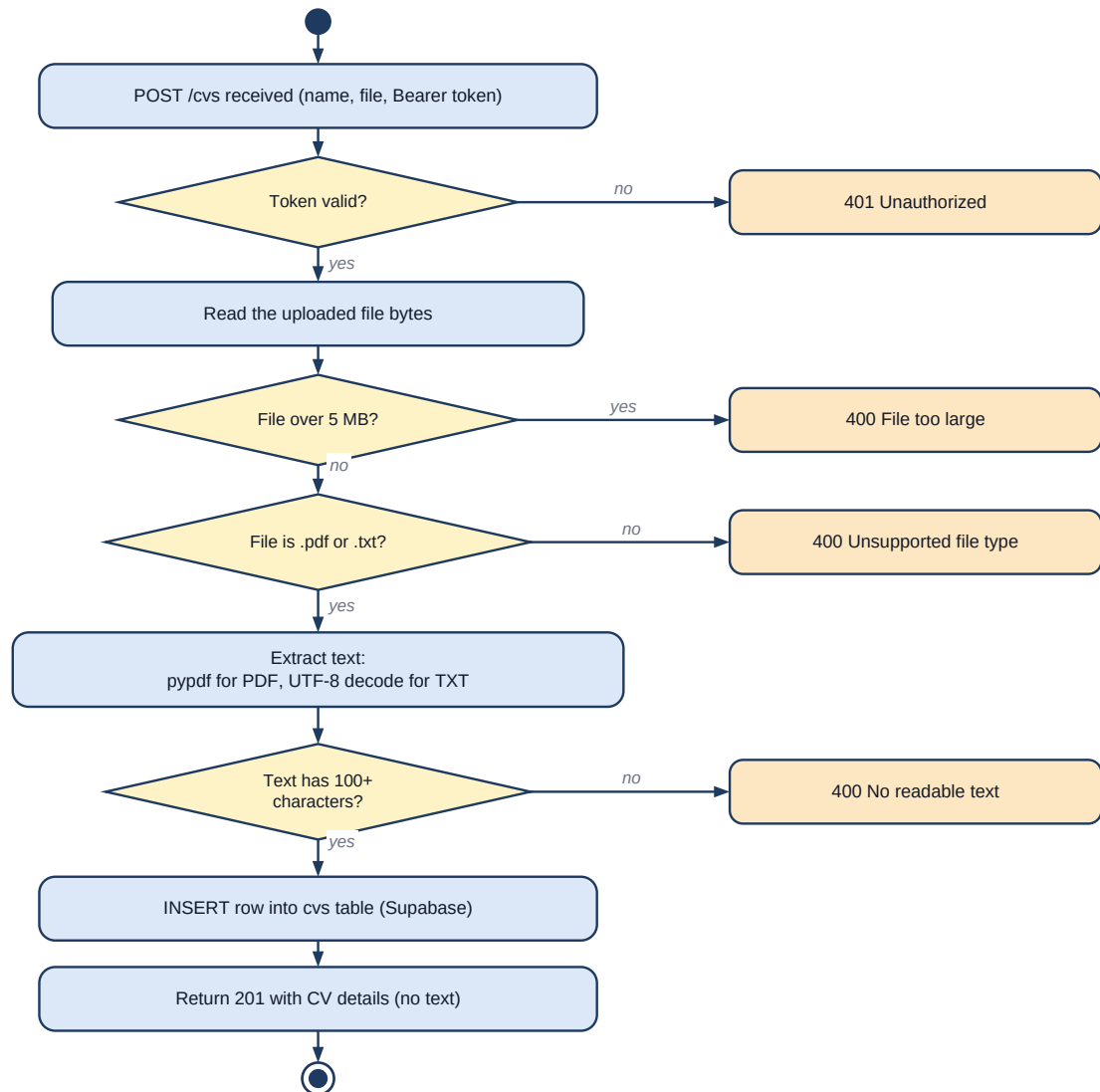


Figure 5: Activity diagram, POST /cvs.

7.2 Run a scan

The first part is a chain of guard checks that fail fast with a clear message. The loop on the right then handles emails one at a time. Any exception inside the loop body is caught, logged, counted as an error, and the loop moves on, so one bad email cannot stop a scan. An email that failed was never saved, so the next scan retries it.

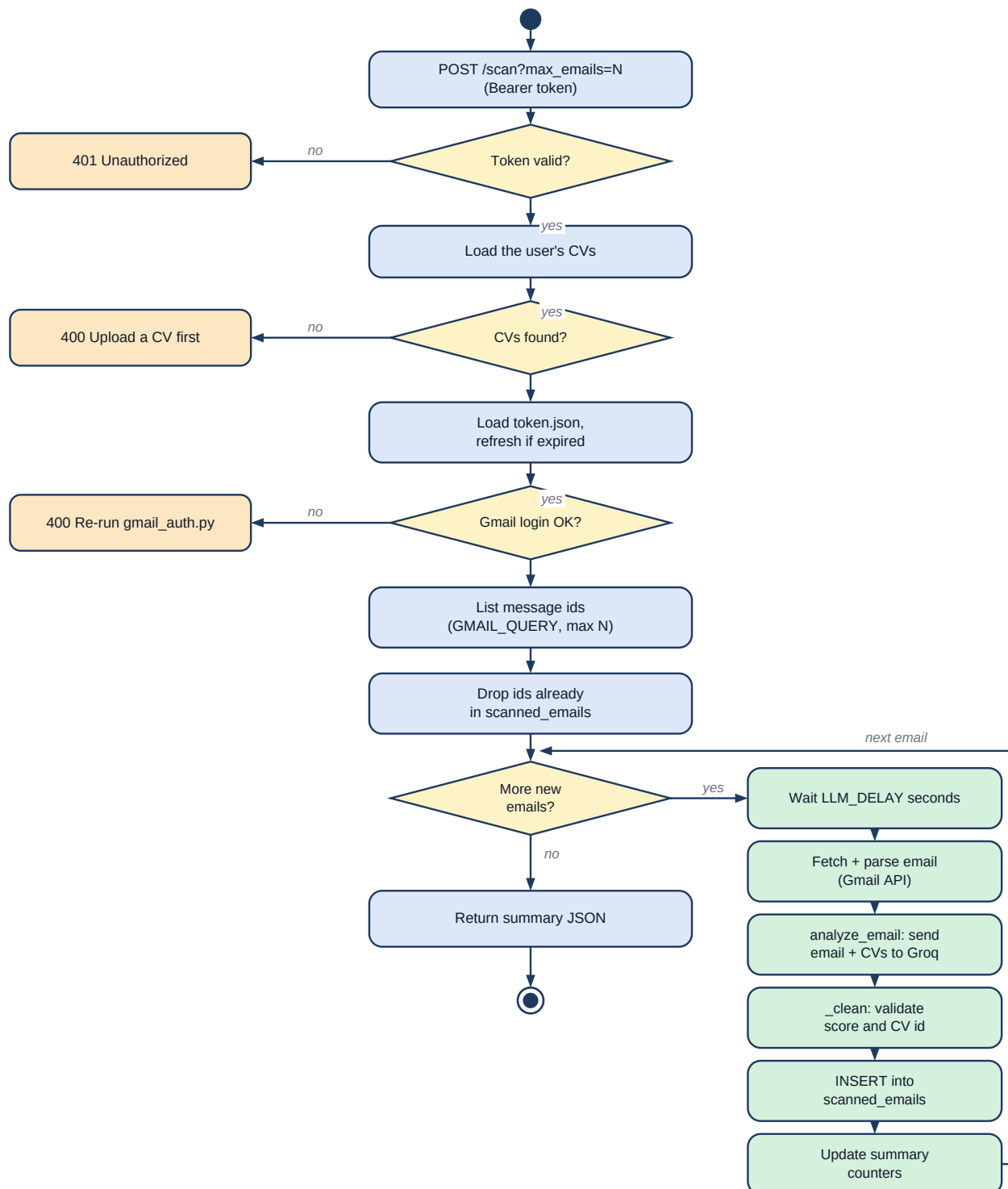


Figure 6: Activity diagram, POST /scan.

7.3 Analyze one email (inside the loop)

This is the merged AI call in detail. Two outcomes matter: an error (the email is skipped and retried later) or a clean, validated dictionary. Nothing the model says reaches the database without passing the validator.

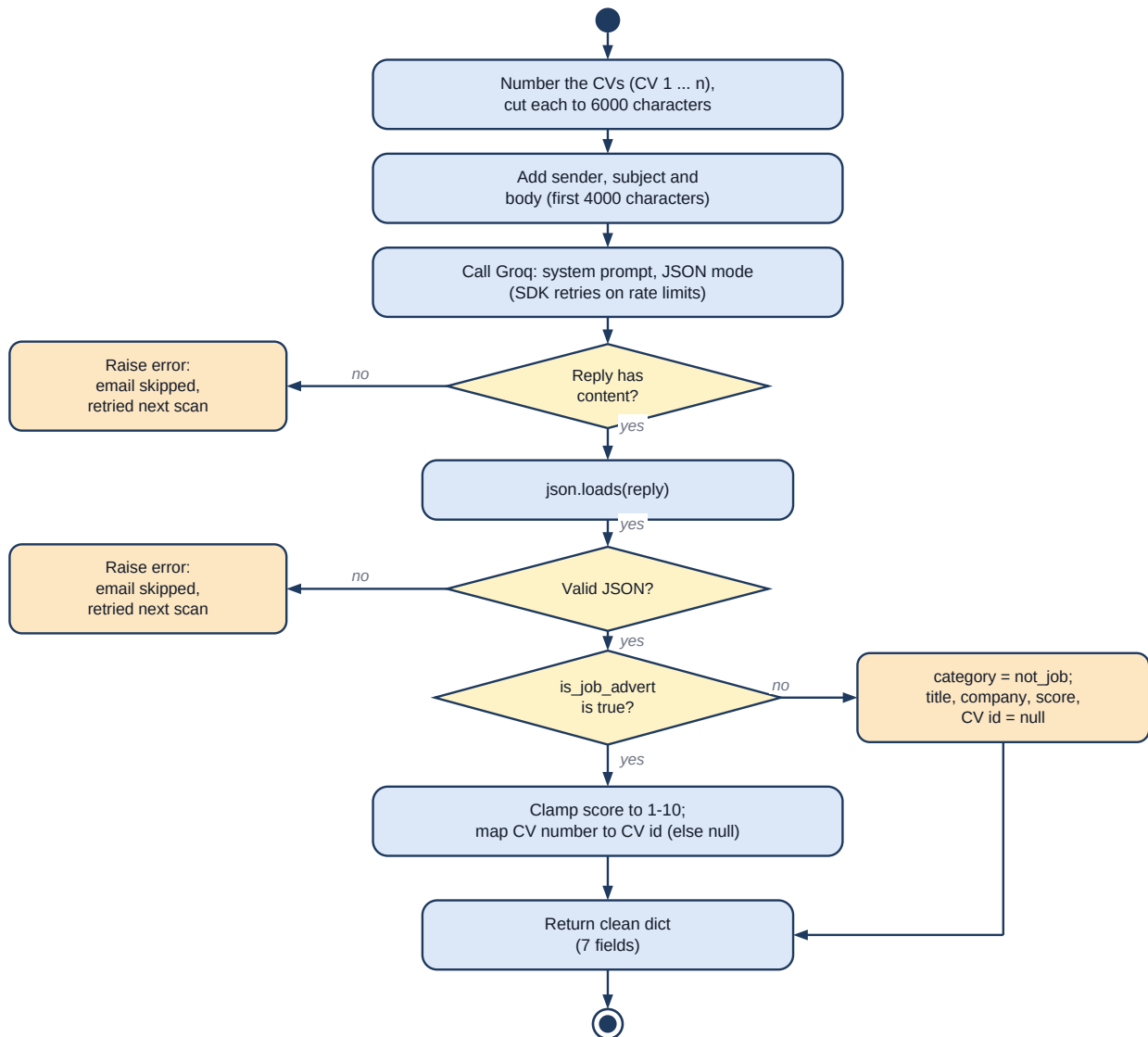


Figure 7: Activity diagram, `analyze_email` and `_clean`.

8. The Logic Behind the Code

This section explains why each module is written the way it is. Full listings are in the Phase 3 build guide; the excerpts below are the parts where the reasoning matters.

8.1 Gmail login: `scripts/gmail_auth.py`

Gmail will not give an app access to a mailbox without the owner's consent. The script runs Google's "installed app" flow: it opens a browser, the user approves, and Google redirects back to a temporary local web server the script started. The result is a credentials object that is saved to `token.json`.

- **Scope = `gmail.readonly`.** The least power that still does the job. A leaked token could read mail but not send or delete it.

- **Two tokens.** The short-lived access token expires in about an hour. The refresh token lets the app obtain new access tokens without asking the user again. The script passes `prompt="consent"` so Google always returns a refresh token.
- **credentials.json vs token.json.** The first identifies the *app* (downloaded from Google Cloud); the second is the *user's permission*. Both are secrets and both are in `.gitignore`.
- **Test mode limit.** While the Google consent screen is in "Testing", only listed test users can log in, and refresh tokens expire after about 7 days. Re-running the script fixes that.

8.2 Gmail client: `app/agent/gmail_client.py`

`get_gmail_service` loads `token.json`, refreshes it if the access token has expired, writes the refreshed token back, and returns a Gmail API handle. Failures become a single custom error, `GmailAuthError`, with an instruction in the message, which the API turns into a readable 400.

Reading an email body is the fiddly part, because a message is a tree of MIME parts, and the same text can appear as plain text, as HTML, or both. The code walks the whole tree, collects both kinds, and prefers plain text; only if none exists does it strip the HTML with BeautifulSoup. Then it removes blank lines to save tokens.

```
def _walk(part, plain, html):
    mime = part.get("mimeType", "")
    data = part.get("body", {}).get("data")
    if data:
        if mime == "text/plain": plain.append(_decode(data))
        elif mime == "text/html": html.append(_decode(data))
    for sub in part.get("parts") or []:
        _walk(sub, plain, html)    # recurse into nested parts
```

- **Base64url decoding.** Gmail returns body data as base64url without padding. The decoder adds the missing "=" characters (`"=" * (-len(data) % 4)`) before decoding, otherwise some messages fail randomly.
- **Received time** comes from Gmail's `internalDate` (milliseconds since 1970), converted to a UTC ISO timestamp for the database.
- **Two-step fetch.** First a cheap list call returns only ids; full messages are fetched only for ids that survive the duplicate filter.

8.3 CV upload: `app/api/cvs.py`

- **Cheapest checks first.** Authentication, size and file type are checked before any parsing happens.
- **Text, not files, is stored.** The AI needs text. Storing the extracted text in `cvs.content_text` means a scan never has to re-parse a PDF.
- **The 100-character rule** rejects image-only PDFs, which would otherwise create a CV that scores everything as a poor match.
- **response_model=CVResponse** filters the reply down to `id`, `name`, `file_name` and `created_at`, so the CV text (personal data) is never sent back to a client.

8.4 Stores: `app/cv_store.py` and `app/email_store.py`

- **Every query includes user_id.** The stores are the single place where data access happens, so user isolation is enforced here, even though the service key could technically read all rows.
- **Duplicate check in one query.** `.in_("gmail_message_id", ids)` fetches all already-scanned ids for the batch at once, then a set difference gives the new ones.
- **Two layers against duplicates.** The application pre-check avoids wasted AI calls; the database unique constraint on (`user_id`, `gmail_message_id`) is the backstop that makes duplicates impossible even if the code is wrong.
- **Ordering.** Results sort by score descending with nulls last (non-job emails have no score), then by received time.

8.5 The merged AI call: `app/agent/analyzer.py`

The system prompt turns the model into a strict reviewer. Its numbered rules each exist for a reason:

Prompt rule	Why it is there
1. Define what counts as a job advert (alerts, digests, recruiter outreach) and what does not (newsletters, receipts, rejections)	Inboxes are full of look-alikes. Without a definition the model scores application receipts as jobs
2. If an email lists several jobs, evaluate only the best-matching one	The table stores one verdict per email, so the model must choose
3. Compare against every CV, pick the best, score 1 to 10, be strict about 9 and 10	Models are generous scorers by default. A strict anchor keeps the 7+ notification threshold meaningful
4. Non-jobs get null fields	Gives the validator a clear contract and keeps junk values out of the table
Output: "ONLY a JSON object with exactly these keys"	Required in JSON mode, which also needs the word JSON in the prompt

Why a validator. Without strict-schema support, the reply is only *probably* well-formed. `_c` Lean treats model output as untrusted input:

```
is_job = data.get("is_job_advert") is True          # anything but a real true is "not a job"
score, cv_id = None, None
if is_job:
    raw = data.get("match_score")
    if isinstance(raw, (int, float)):
        score = max(1, min(10, int(raw)))           # clamp to the database's allowed range
    n = data.get("best_cv_number")
    if isinstance(n, int) and 1 <= n <= len(cvs):
        cv_id = cvs[n - 1]["id"]                   # number -> real id, or null if out of range
```

- **Clamping** protects the database check constraint (match_score between 1 and 10): a model answer of 11 becomes 10 instead of crashing the insert.
- **Number-to-id mapping** means a hallucinated CV number simply yields null rather than a foreign-key error.
- **No temperature parameter.** Some newer models reject it; leaving it out keeps the code portable across providers.
- **max_retries=6.** On a rate-limit response (429) the SDK waits and retries by itself, so free-tier limits slow a scan instead of failing it.

8.6 Orchestration: `app/agent/scanner.py`

The scanner is the only module that knows the order of work. Its core is small:

```
cvs = cv_store.get_cvs_with_text(user_id)
if not cvs: raise NoCVsError(...)                # fail fast, before touching Gmail or paying for AI
ids = list_message_ids(service, settings.gmail_query, max_emails)
existing = email_store.get_existing_message_ids(user_id, ids)
new_ids = [i for i in ids if i not in existing]

for msg_id in new_ids:
    time.sleep(settings.llm_delay_seconds)         # stay under rate limits
    try:
        email = get_message(service, msg_id)
        analysis = analyze_email(email, cvs)
        email_store.save_scanned_email(user_id, email, analysis)
    except Exception:
        logger.exception(...); summary["errors"] += 1; continue    # isolate failures
    # update counters; a job counts as "above threshold" only when score > 7
```

- **Fail fast.** Missing CVs or a missing Gmail login are detected before any slow or billable work.
- **Error isolation.** The try/except sits inside the loop, so one bad email costs one email, not the whole scan.
- **Save after success only.** The save is the last step of the try block. A failure earlier leaves no row, which is exactly what makes retry work.
- **Threshold is strictly greater than 7** (scores 8 to 10), matching the notification rule that Phase 4 will use. The value lives in `notify_threshold`.

- **The summary** (fetched, already_scanned, analyzed, job_adverts, above_threshold, errors) is the quickest health check of a scan.

8.7 Endpoints: app/api/scan.py

The router contains no business logic. NoCVsError and GmailAuthError are translated into HTTP 400 responses with actionable messages; everything else is a plain 500 from FastAPI, with the full traceback in the log. The user id always comes from the verified token (current_user["id"]), never from request parameters, so a caller cannot scan or read another user's data.

9. Data Touched in the Database

Table	Columns used	Written by	Notes
cvs	user_id, name, file_name, content_text	POST /cvs	content_text is read by every scan; deleting a user cascades to their CVs
scanned_emails	user_id, gmail_message_id, subject, sender, received_at, is_job_advert, category, job_title, company, match_score, best_matching_cv_id, reasoning	scanner	notified stays false until Phase 4. Unique on (user_id, gmail_message_id). match_score checked to 1 to 10

10. Setup Summary

#	Step	Detail
1	Install packages	pip install openai pypdf python-multipart beautifulsoup4 google-api-python-client google-auth-oauthlib google-auth-httplib2, then pip freeze
2	Groq key and config	console.groq.com, API Keys, create key. Add LLM_API_KEY, LLM_BASE_URL, LLM_MODEL, GMAIL_QUERY to .env; add the fields to config.py; remove any old openai_api_key field
3	Google Cloud	New project, enable Gmail API, consent screen (External, Testing), add your Gmail under Test users, create a Desktop OAuth client, save as backend/credentials.json
4	Connect Gmail	python scripts\gmail_auth.py from backend; approve in the browser; token.json appears
5	Write the code	cv_store, email_store, schemas/cv, api/cvs, agent/gmail_client, agent/analyzer, agent/scanner, api/scan; register the routers in main.py
6	Test	Follow the checklist in section 11
7	Commit	git status (no .env, credentials.json or token.json), then add, commit, push

```
LLM_API_KEY=gsk_your_groq_key
LLM_BASE_URL=https://api.groq.com/openai/v1
LLM_MODEL=llama-3.3-70b-versatile
GMAIL_QUERY=newer_than:3d
```

11. Verification Checklist

#	Test	Expected result
1	Log in and Authorize in /docs	Token accepted; GET /cvs returns 200 and an empty list
2	POST /cvs with each CV (name + PDF)	201 for each; GET /cvs lists them; Supabase cvs table shows extracted text
3	POST /cvs with a scanned PDF	400 "Almost no text found"
4	Run python scripts\gmail_auth.py	token.json created in backend
5	One-line Gmail test (list 3 subjects)	Three subject lines from the inbox are printed
6	POST /scan?max_emails=5	200 with a summary; terminal shows "JOB score=..." lines
7	GET /scan/results	Ranked list, highest score first; non-jobs at the bottom with null score

#	Test	Expected result
8	POST /scan again	already_scanned rises and analyzed is 0 for those emails (proves duplicate protection)
9	GET /scan/results?min_score=8	Only emails scoring 8 or more
10	Read the reasoning of several results against the real emails	Scores feel sensible. If not, tune the prompt rules in analyzer.py
11	git status before committing	No .env, credentials.json or token.json listed

12. Troubleshooting

Symptom	Cause and fix
ModuleNotFoundError: No module named app.schemas.cv	File is misnamed (for example cvs.py instead of cv.py) or in the wrong folder. Check with: <code>dir app\schemas</code> . Also add an empty <code>__init__.py</code> there
401 on /cvs or /scan	No token sent. Log in, copy the access_token, click Authorize, paste only the token. Swagger forgets it on page reload
401 on /auth/login right after signup	Email or password differs from signup (Swagger pre-fills placeholders). Retype both exactly
Access blocked / Error 403 access_denied (Google)	Your address is not under Test users on the OAuth consent screen. Add it, then re-run <code>gmail_auth.py</code>
token.json not found	Run <code>python scripts\gmail_auth.py</code> from backend
Gmail login expired	Google test-mode refresh tokens last about 7 days. Re-run <code>gmail_auth.py</code>
Gmail API has not been used (403)	Gmail API not enabled in the same Google Cloud project as <code>credentials.json</code>
Field required: llm_api_key	Missing or misnamed .env entry, or the server was not fully restarted
Field required: openai_api_key	Old field still in <code>config.py</code> . Delete it
401 Invalid API Key (Grog)	Wrong or truncated key. Copy the full <code>gsk_</code> value without quotes
429 rate limit repeating	Free-tier cap. Raise <code>LLM_DELAY_SECONDS</code> , lower <code>max_emails</code> , or wait for the limit to reset
model_decommissioned or model not found	Grog retires models. Choose a current one at console.groq.com/docs/models and set <code>LLM_MODEL</code>
Almost no text found on CV upload	Scanned or image PDF. Export a text-based PDF or upload a .txt file
422 on POST /cvs	<code>python-multipart</code> missing: <code>pip install python-multipart</code>
Occasional JSON errors in the log	The model returned malformed JSON for one email. It is skipped and retried next scan
Scores all too high or too low	Tune rule 3 of the system prompt, or try a larger model

13. Known Limitations and Notes

- **One verdict per email.** A digest listing ten jobs is scored on its single best match. Splitting digests into one row per job is a later improvement.
- **Single mailbox.** The local `token.json` belongs to one Google account. Per-user Gmail connections arrive in Phase 6.
- **Weekly re-login.** While the Google app is in Testing mode, the Gmail login must be refreshed about every 7 days.
- **Privacy.** Email text and CV text are sent to Groq for analysis. Review the provider's data policy if that matters to you. Only the first 4,000 characters of an email and 6,000 of each CV are sent.
- **Free-tier limits.** Groq limits requests and tokens per minute and per day. The delay and automatic retries soften this, but large scans may still hit a daily cap.
- **Scores are opinions.** A 70B model's scoring depends on the prompt and CV quality. Calibrate it against emails you can judge yourself before trusting the 7+ threshold.

- **Image-only PDFs** are rejected; there is no OCR.
- **Moving to OpenAI later** needs only three .env values (LLM_API_KEY, LLM_BASE_URL, LLM_MODEL); no code changes.

14. Outcome and Next Phase

Phase 3 is complete when a manual scan of the real inbox returns sensible scores, a second scan processes nothing twice, and results can be listed in ranked order. At that point the hardest part of the product, turning raw email into a scored, CV-matched verdict, works and is saved.

Phase 4 (Telegram notifications) is next and is the first "it works" moment: after each scan, every job scoring above 7 is sent to your phone, and its notified flag is set so it is never sent twice. It uses the `user_settings` table that already exists for the Telegram chat id and threshold.